

Миша Филитов

АКОС 5

System V AMD64 ABI

НИУ ВШЭ ФКН ПМИ 2026

Секции

- ❖ **.text** - код программы - R_X
- ❖ **.data** - изменяемые данные - RW_
- ❖ **.rodata** - неизменяемые данные - R__
- ❖ **.bss** - зануленные изменяемые данные - RW_
- ❖ Данные находятся в самом бинаре
- ❖ **.bss** место выделяется при старте программы

```
3 // .data
4 int initialized = 42;
5 // .bss
6 int buffer[1000];
7
8 int main() {
9     // txt - локальная переменная
10    // "some text" - .rodata
11    char *txt = "some text";
12
13    // "%s\n", "%d\n" - .rodata
14    printf("%s\n", txt);
15    printf("%d\n", initialized);
16    printf("%d\n", buffer[0]);
17
18    return 0;
19 }
```


Данные в секциях

- ❖ `.byte` = 1 байт
- ❖ `.word` = 2 байта
- ❖ `.int` = 4 байта
- ❖ `.quad` = 8 байт
- ❖ `.skip N` = зарезервировать N байт
- ❖ `.skip 4 * N` = N элементов по 4 байта
- ❖ `.skip N, V` = зарезервировать N байт и заполнить каждый байт значением V
- ❖ `.ascii "abc"` = 3 байта
- ❖ `.asciz "abc"` = 4 байта, включая `'\0'`

```
1  .data
2
3  // 8 байт
4  var:
5      .quad 42
6
7  // 5 * 4 = 20 байт
8  array32:
9      .int 10, 20, 30, 40, 50
10
11 .rodata
12 // 4 * 8 = 32 байта
13 array64:
14     .quad 100, 200, 300, 400
15
16 .data
17 // 5 байт
18 bytes:
19     .byte 1, 2, 3, 4, 5
20
21 // 7 байт: 6 символов + '\0'
22 text:
23     .asciz "Hello!"
```

Адрессация

$[base + index * scale + displacement]$

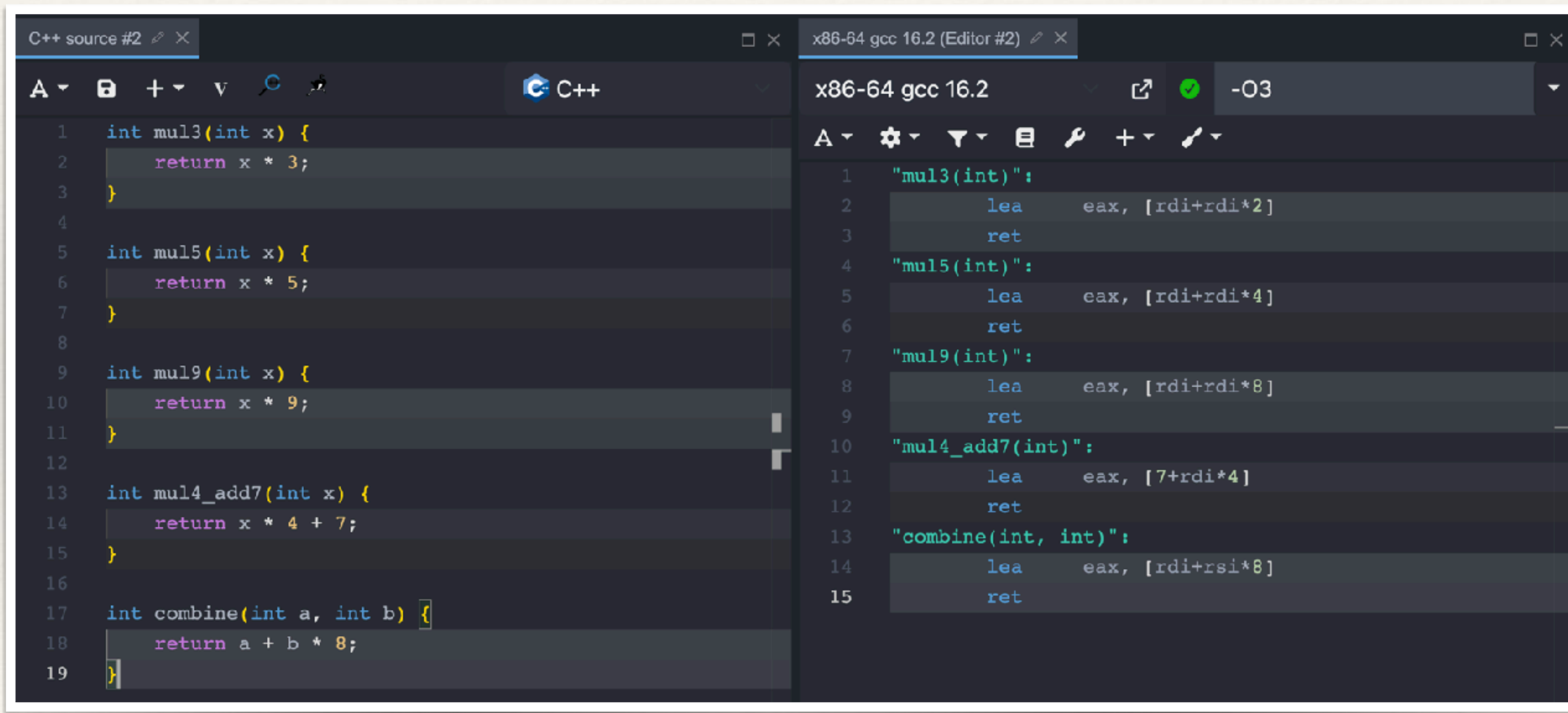
- ❖ base - базовый регистр
- ❖ index - индексный регистр
- ❖ scale - множитель: 1, 2, 4 или 8
- ❖ displacement - константное смещение

```
1 // прочитать 8 байт из памяти по адресу rax
2 mov rax, [rax]
3
4 // прочитать 8 байт из памяти по адресу rax + 8
5 mov rax, [rax + 8]
6
7 // прочитать 8 байт из памяти по адресу rax + rcx
8 mov rax, [rax + rcx]
9
10 // прочитать 8 байт из памяти по адресу rax + rcx * 4
11 mov rax, [rax + rcx * 4]
12
13 // прочитать 8 байт из памяти по адресу rax + rcx * 8 + 16
14 mov rax, [rax + rcx * 8 + 16]
```

<https://godbolt.org/z/cs7j3KjGo>

lea - load effective address

$[\text{base} + \text{index} * \text{scale} + \text{displacement}]$



```
C++ source #2
1 int mul3(int x) {
2     return x * 3;
3 }
4
5 int mul5(int x) {
6     return x * 5;
7 }
8
9 int mul9(int x) {
10    return x * 9;
11 }
12
13 int mul4_add7(int x) {
14    return x * 4 + 7;
15 }
16
17 int combine(int a, int b) {
18    return a + b * 8;
19 }

x86-64 gcc 16.2 (Editor #2)
x86-64 gcc 16.2 -O3
1 "mul3(int)":
2     lea     eax, [rdi+rdi*2]
3     ret
4 "mul5(int)":
5     lea     eax, [rdi+rdi*4]
6     ret
7 "mul9(int)":
8     lea     eax, [rdi+rdi*8]
9     ret
10 "mul4_add7(int)":
11    lea     eax, [7+rdi*4]
12    ret
13 "combine(int, int)":
14    lea     eax, [rdi+rsi*8]
15    ret
```

<https://godbolt.org/z/h9db8ff48>

RIP-relative addressing

```
5  main:
6      // [ base + index*scale + disp ]
7      mov     qword ptr [var + rip], 500
8      mov     rdi, [var + rip]
9      call    writei64
10
11     call    finish
12
13  .data
14  var:
15      .quad   42
16
17  // .byte - 1 байт
18  // .word - 2 байта
19  // .int  - 4 байта
20  // .quad - 8 байт
```

x86-64 gcc 16.2

Program returned: 0
Program stdout
500

<https://godbolt.org/z/3T87a9rGs>

RIP-relative addressing

```
4      .text
5  var:
6      .quad    42
7  main:
8      // [ base + index*scale + disp ]
9      mov     qword ptr [var + rip], 500
10     mov     rdi, [var + rip]
11     call    writei64
12
13     call    finish
```

```
0000000000001169 <var>:
1169:    2a 00                sub     al,BYTE PTR [rax]
116b:    00 00                add     BYTE PTR [rax],al
116d:    00 00                add     BYTE PTR [rax],al
...
0000000000001171 <main>:
1171:    48 c7 05 ed ff ff ff  mov     QWORD PTR [rip+0xffffffffffffffed],0x1f4
1178:    f4 01 00 00
117c:    48 8b 3d e6 ff ff ff  mov     rdi,QWORD PTR [rip+0xffffffffffffffe6]
1183:    e8 b8 00 00 00        call    1240 <writei64>
1188:    e8 53 00 00 00        call    11e0 <finish>
118d:    0f 1f 00              nop     DWORD PTR [rax]
```

$0xffffffffffffffed = -19$

$0x1169 - 0x117c = 0xffffffffffffffed$

STACK

stack

- ❖ Растет в сторону уменьшения адресов
- ❖ Адрес вершины стека - **rsp**
- ❖ Обычно 8мб (`ulimit -s`)
- ❖ При переполнении SIGSEGV
- ❖ Используется для локальных переменных, адресов возврата, сохранения значений регистров
- ❖ **push** - положить на стек, **pop** - забрать

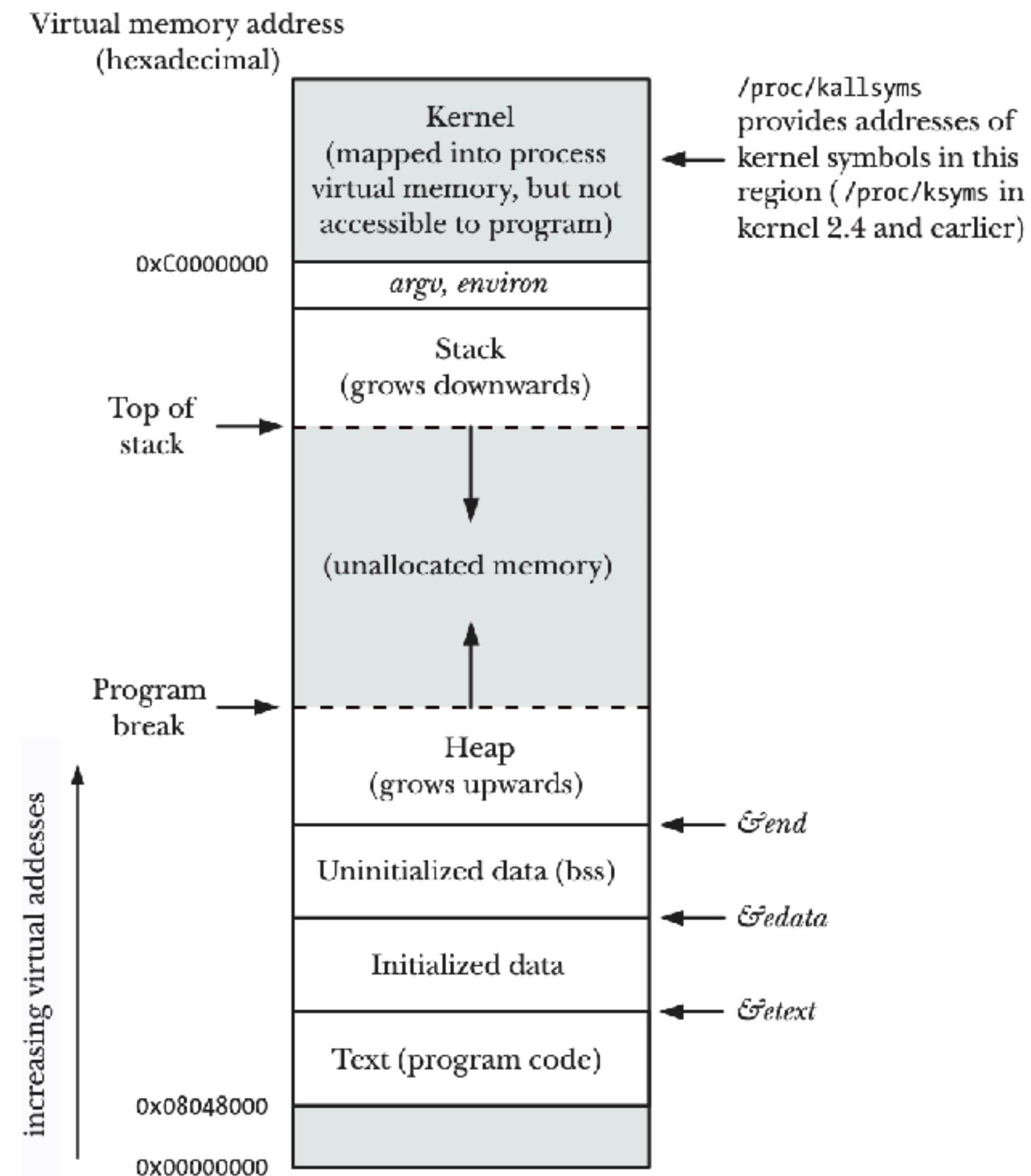


Figure 6-1: Typical memory layout of a process on Linux/x86-32

Работа со стеком

- ❖ `push 42` - положить 42 на вершину и изменить `rsp`
- ❖ `pop rax` - забрать значение с вершины стека в `rax` и изменить `rsp`
- ❖ `sub rsp 16` - выделить на стеке 16 байт
- ❖ `mov [rsp], rax` - положить на вершину `rax`
- ❖ `mov qword ptr [rsp], 42` - положить 42 на вершину

```
4  main:
5      // [ return   xxxxx ]
6      // ^rsp
7      call    readi64
8      push    rax
9      // [ rax1, return   xxxxx ]
10     // ^rsp
11
12     call    readi64
13     sub     rsp, 8
14     // [ yyyyy, rax1, return   xxxxx ]
15     // ^rsp
16     mov     [rsp], rax
17     // [ rax2, rax1, return   xxxxx ]
18     // ^rsp
19
20     // Выведем то, что лежит под rsp (последнее введенное число)
21     mov     rdi, [rsp]
22     call    writei64
23     mov     rdi, [rsp + 8]
24     call    writei64
25
26     pop     rdi
27     call    writei64
28     // [ rax2, rax1, return   xxxxx ]
29     // ^rsp
30
31     pop     rdi
32     call    writei64
33     // [ rax2, rax1, return   xxxxx ]
34     // ^rsp
35     call    finish
```


Вызов функций

- ❖ Нельзя вызвать из разных мест
- ❖ Рекурсивные вызовы??
- ❖ Адрес возврата зашит в функцию

```
4  add_numbers:
5      add    rdi, rax
6      jmp    after_add
7
8  main:
9      mov    rdi, 5
10     mov    rax, 6
11
12     jmp    add_numbers
13
14  after_add:
15     // some code
```

Вызов функций

```
4  add_numbers:
5      add     rdi, rax
6      pop     r12
7      jmp     r12
8
9  main:
10     mov     rdi, 5
11     mov     rax, 6
12
13     lea     r12, [rip + after_add]
14     push    r12
15     jmp     add_numbers
16
17 after_add:
18     // some code
```

<https://godbolt.org/z/3Tzb8rxKx>

```
4  add_numbers:
5      add     rdi, rax
6      ret
7
8  main:
9      mov     rdi, 5
10     mov     rax, 6
11
12     call    add_numbers
13
14 after_add:
15     // some code
```

<https://godbolt.org/z/8dv8z7es3>

Соглашение о вызовах

System V AMD64 ABI

*ABI - Application Binary Interface

System V AMD64 ABI

- ❖ Первые 6 аргументов передаются в регистрах **rdi, rsi, rdx, rcx, r8, r9**
- ❖ Аргументы с плавающей точкой используют **xmm0–xmm7**
- ❖ Прочие параметры передаются в стеке, причём лежат в памяти «по порядку» (адрес увеличивается вместе с номером аргумента)
- ❖ Память под переданные через стек аргументы освобождает вызывающая функция
- ❖ Возвращаемое значение в регистре **rax** (а 128-битное — в паре **rdx:rax**)
- ❖ callee-saved регистры: **rbx, rsp, rbp, r12–r15** - вызываемая функция должна вернуть их с прежними значениями
- ❖ caller-saved регистры: **rax, rcx, rdx, rsi, rdi, r8–r11** - вызывающая функция не может рассчитывать, что их значения сохранятся после вызова
- ❖ Непосредственно перед вызовом **rsp** должен быть выровнен на 16 байт

Пролог/эпилог

- ❖ После вызова функции **rsp** не должен измениться
- ❖ После вызова функции стек не выровнен по 16 (из-за **push retaddr**)

```
1      .intel_syntax noprefix
2      .global main
3
4  main:
5      // --- Пролог ---
6      push    rbp
7      mov     rbp, rsp
8      // -----
9
10
11     // --- Всякий мудрый код ---
12
13     // .....
14     // .....
15     // .....
16     // .....
17
18     // --- Конец всякого мудрого кода ---
19
20     // --- Эпилог ---
21     mov     rsp, rbp
22     pop     rbp
23     // -----
24
25     xor     eax, eax
26
27     ret
28
```



```

8 read_and_add:
9 // ----- пролог ----- (сохраняем значения регистров, которые мы не должны менять)
10 // Текущее состояние стека (слева-верх, справа низ)
11 // [ return   xxxxx xxxxx xxxxx]
12 //   ^rsp      ^rbp
13 push  rbp      // сохраняем старое значение rbp
14 mov   rbp, rsp // сохраняем значение rsp в rbp, чтобы дальше отсчитывать адреса от rbp, так как rsp будет двигаться
15
16 // [ oldrbp return xxxxx]
17 //   ^rbp,rsp
18 // ----- конец пролога -----
19
20 sub    rsp, 16 // выделим место под 2 числа - 16 байт. rsp всегда на вершине стека,
21           // так как стек растёт в сторону уменьшения адресов, то нужно вычитать из rsp
22
23 // Текущее состояние стека (слева верх, справа низ)
24 // |---|----- наши два числа, для которых мы сделали sub 16. А B просто для наглядности
25 // | | |----- старое значение rbp, которое мы запушили
26 // | | |----- адрес возврата
27 // [ A  B  oldrbp return xxxxx]
28 //   ^rsp  ^rbp
29
30 call   readi64
31 mov    [rbp - 8], rax // кладем в "B". Так как хотим положить "левее" rbp, то знак минус
32           // аналогично могли бы сделать mov [rsp + 8], rax, попали бы в ту же ячейку
33           // но это не очень удобно, потому что rbp статичный, а rsp движается (от пушей, например)
34           // поэтому лучше отсчитывать от rbp
35
36 call   readi64
37 mov    [rbp - 16], rax // аналогично кладем в "A"
38
39 xor     rax, rax // зануляем rax
40 add     rax, [rbp - 16] // прибавляем наши A и B к rax
41 add     rax, [rbp - 8]
42
43 // ----- эпилог ----- (возвращаем сохраненные значения регистров обратно)
44 mov     rsp, rbp
45 // Текущее состояние стека (слева верх, справа низ)
46 // [ A  B  oldrbp return xxxxx]
47 //   ^rsp,rpb
48 pop     rbp
49 // Текущее состояние стека (слева верх, справа низ)
50 // [ A  B  oldrbp return xxxxx xxxxx xxxxx]
51 //   ^rsp      ^rbp
52 // ----- конец эпилога -----
53 ret

```

```

56 .global main
57 main:
58     push    rbp
59     mov     rbp, rsp
60     call    read_and_add
61
62     mov     rdi, rax
63     call    writei64
64
65     mov     rsp, rbp
66     pop     rbp
67     call    finish

```

Следующее занятие

- ❖ Вещественные числа
- ❖ SIMD